

EXPERIMENT NO.1

Aim: To develop and Establish local Blockchain using Truffle

Theory:

Q. What is Truffle?

Truffle is a “world-class development environment, testing framework, and asset pipeline for blockchains using the Ethereum Virtual Machine (EVM), aiming to make life as a developer easier.” Truffle is a tool that makes the life of a Blockchain Developer easy. You can easily create, compile, deploy and test your smart contract by using a simple truffle command.

Q. How to Install Truffle Suite?

Following are the system requirements before installing the truffle suite on your system:

1. NodeJS v12 or later
2. Install the latest version of Visual Studio

Run the following command in the command prompt

```
npm install -g truffle
```

Code language: Bash (bash)

Once done, truffle should now be available on your system. If you're learning truffle on codedamn's web3 learning path – you do not need to install or set up anything as the interactive labs, come with the truffle setup already.

Q. How to check whether Truffle is installed or not?

Run the following command in the command prompt:

```
truffle version
```

Code language: Bash (bash)

If you are seeing something like the below it means truffle is installed
(note that your versions could vary):

```
Truffle v5.5.0 (core: 5.5.0) Ganache v7.0.1
```

```
Solidity v0.5.16 (solc-js) Node v14.17.2
```

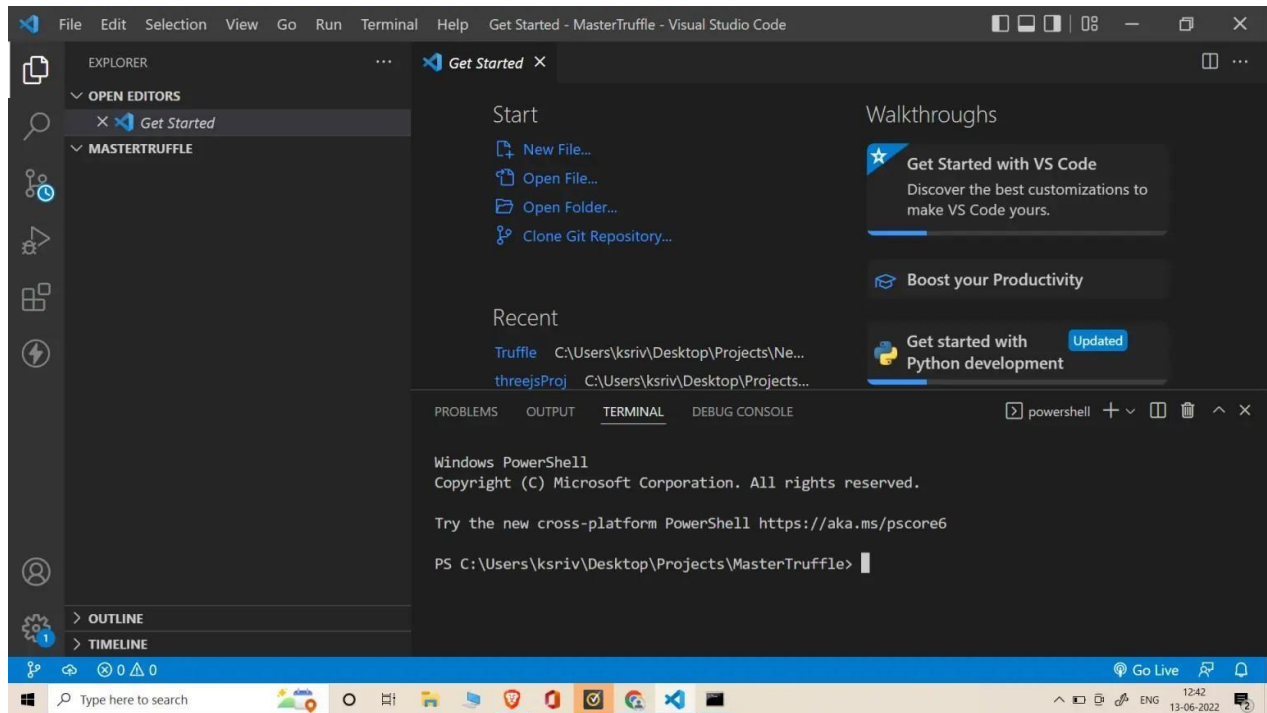
```
Web3.js v1.5.3
```

Code language: Bash (bash)

Q. How to use Truffle for a Smart Contract?

Use the following steps to start using truffle for building a smart contract: Install Visual Studio Code.

Create a folder by MasterTruffle or whatever you wish. Open your terminal in VS Code



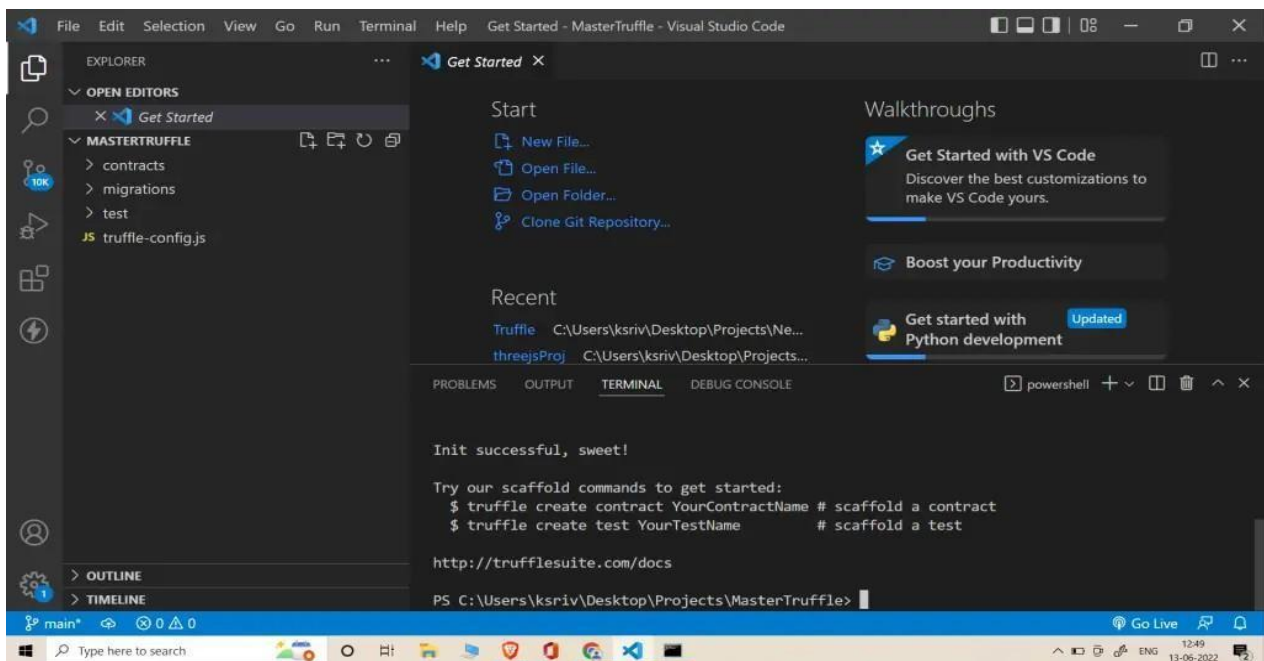
Creating MasterTruffle Folder in VS code Run the following command in the terminal:

truffle init

Code language: Bash (bash)

This will create a folder structure as below:

1. Contract Folder – In this, you will be creating your smart contracts.
2. Migrations Folder – This will contain the migration script to migrate your contract on the Blockchain.
3. Test – This will contain the test file for your smart contract.
4. truffle-config.js – This is the configuration file for setting the various things like the solidity compiler version, the blockchain at which you want to deploy your smart contract, etc.



Folders are created after the `truffle init` command

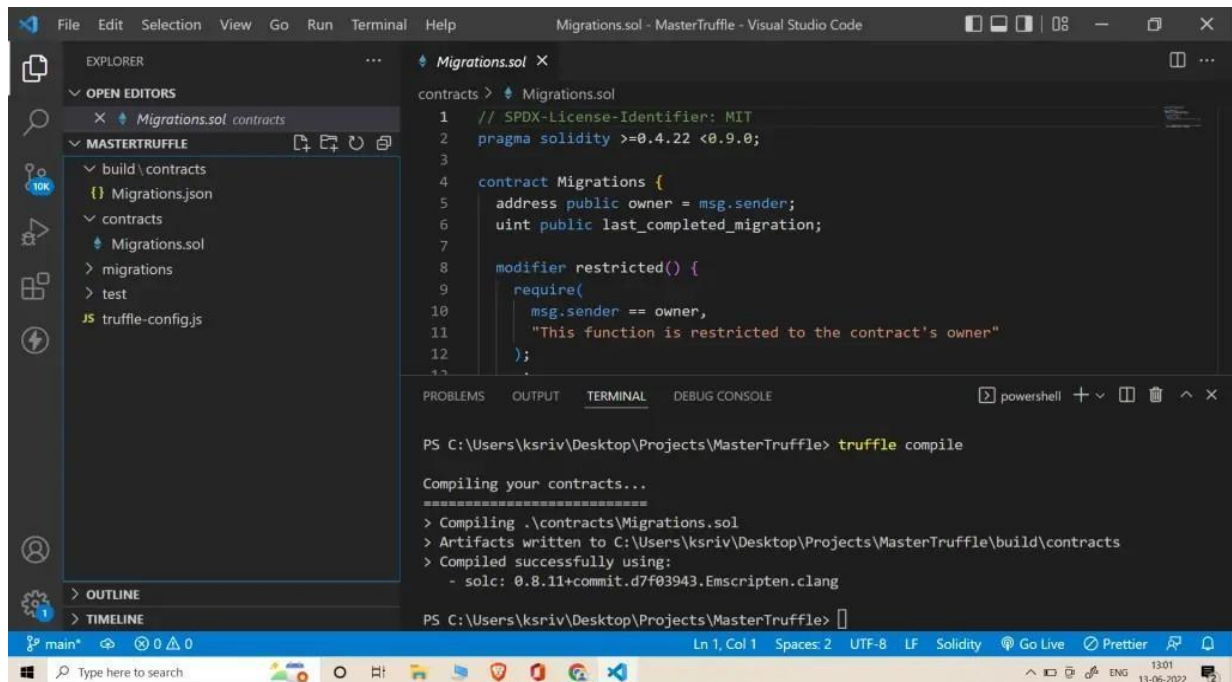
Q. How to compile a smart contract?

After coding your smart contract, run the following command in the command prompt:

`truffle compile`

Code language: Bash (bash)

This will compile all the smart contracts present in the contracts folder in one go. Once your smart contract will get compiled; a build folder will be created automatically which will consist of the Bytecode and ABI of your smart contract in a JSON format.



The screenshot shows the Visual Studio Code interface with a project named 'MasterTruffle'. The Explorer panel on the left shows the file structure: 'contracts' folder containing 'Migrations.sol', 'migrations' folder, 'test' folder, and 'truffle-config.js'. The main editor shows the content of 'Migrations.sol', which includes a Solidity pragma statement and a contract definition. The Terminal panel at the bottom shows the command 'truffle compile' being executed, followed by the output indicating successful compilation of the contract.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.4.22 <0.9.0;
3
4 contract Migrations {
5     address public owner = msg.sender;
6     uint public last_completed_migration;
7
8     modifier restricted() {
9         require(
10             msg.sender == owner,
11             "This function is restricted to the contract's owner"
12         );
13     }
14 }
```

```
PS C:\Users\ksriv\Desktop\Projects\MasterTruffle> truffle compile

Compiling your contracts...
=====
> Compiling .\contracts\Migrations.sol
> Artifacts written to C:\Users\ksriv\Desktop\Projects\MasterTruffle\build\contracts
> Compiled successfully using:
   - solc: 0.8.11+commit.d7f03943.Emscripten.clang

PS C:\Users\ksriv\Desktop\Projects\MasterTruffle>
```

Q. How to Deploy the Smart Contract on The Blockchain Using Ganache?

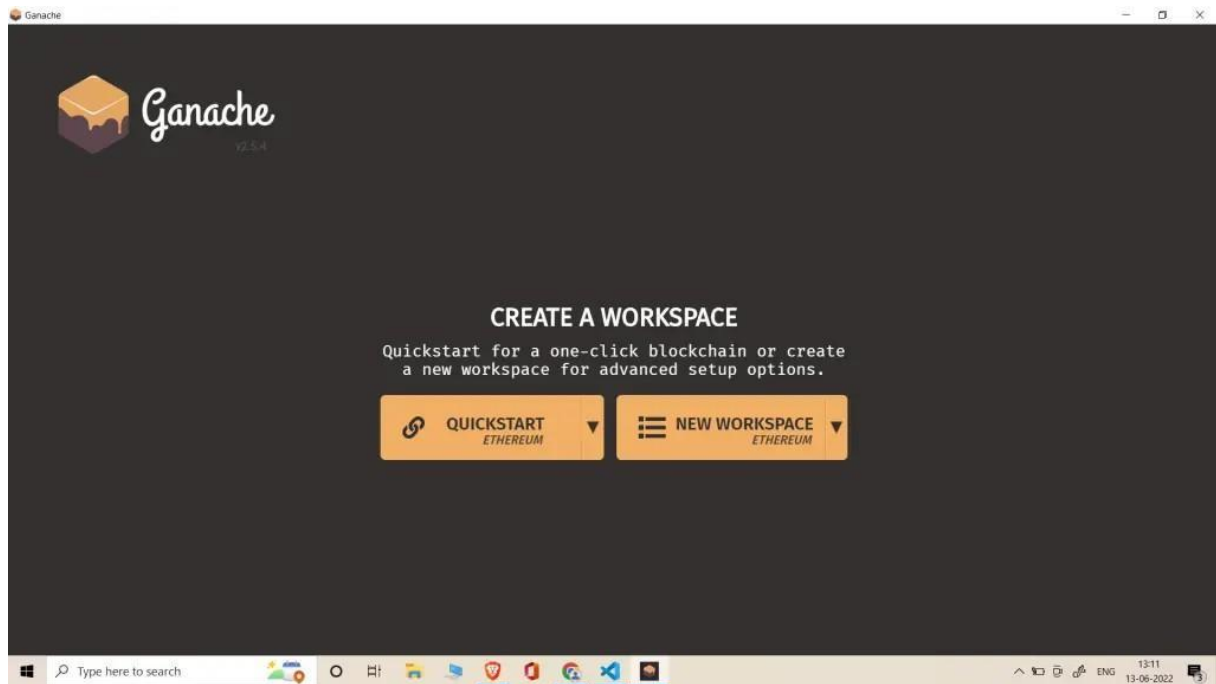
To deploy/migrate the smart contract on Blockchain we need to decide on the Blockchain on which we want to deploy our smart contract. In this blog, I am going to use Ganache to deploy our smart contract.

Introduction to Ganache

Ganache is a local blockchain simulator that runs on your system. The best part, it's completely free and you do not need to spend any money to deploy your smart contract. Ganache provides us with free testing ethers which we can use to test our smart contract.

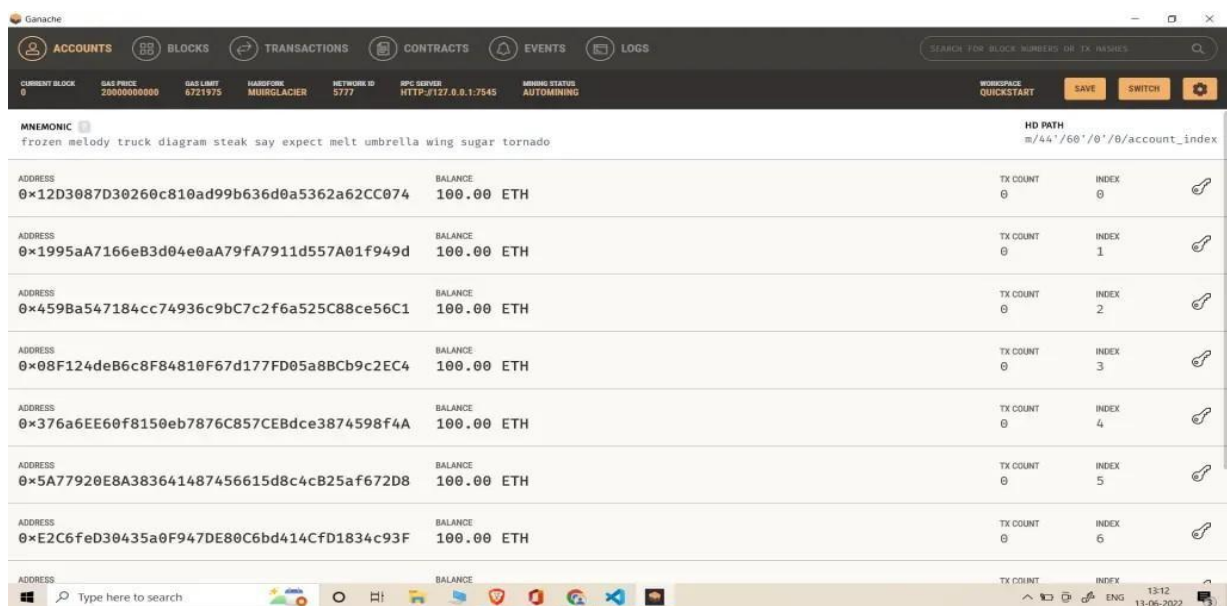
Q. How to Install Ganache?

- Go to this site – [Truffle Suite](#) and click on Download.



After installation

- Then, click on “Quickstart”. This will start your ganache blockchain.
- The ganache blockchain consists of various accounts & each account balance is 100 ethers.
- These ethers are test ethers for testing your smart contract before you deploy your smart contract on the Ethereum Mainnet (Real Blockchain).
- Let us check our migration script available in the migration folder.



- As you will notice, the migration file for the Migration.sol smart contract is starting with a number i.e. 1.
- Whenever you name your migration script always start with a number.

- This is to tell truffle in which order the smart contracts will be deployed.
- So, let's say you have created 3 smart contracts a.sol, b.sol, c.sol. And you want to deploy your smart contract in the following order c,a,b.
- In this case, your migration files will be named 1_c.js, 2_a.js, and 3_b.js.

```

JS 1_initial_migration.js X
migrations > JS 1_initial_migration.js > ...
1  const Migrations = artifacts.require("Migrations");
2
3  module.exports = function (deployer) {
4    deployer.deploy(Migrations);
5  };

```

Migration file for our migration.sol smart contract in truffle suite

1. Whenever you want to create a migration file for your smart contract, you should have your contract name in the artifacts.require("Your smart contract name").

contract Demo {}

Code language: JavaScript (javascript)

2. The migration file for the above smart contract will look like the below.

```

1  const Demo = artifacts.require("Demo");
2
3  module.exports = function (deployer) {
4    deployer.deploy(Demo);
5  };

```

Once we have created our migration file, let's make changes to our truffle configuration file. In the truffle-config.js file search for development. Comment out the development part by selecting it and then press ctrl+/.

Change the **port number to 7545** which is the port for the ganache UI that we have installed.

```

JS truffle-config.js X
JS truffle-config.js > [?] <unknown> > networks
38  // Useful for testing. The `development` name is special - truffle uses it
39  // if it's defined here and no other network is specified at the command line
40  // You should run a client (like ganache-cli, geth or parity) in a separate
41  // terminal if you use this network and you must also set the `host`, `port` and
42  // `network_id` options below to some value.
43  //
44  development: {
45    host: "127.0.0.1", // Localhost (default: none)
46    port: 7545, // Standard Ethereum port (default: none)
47    network_id: "*", // Any network (default: none)
48  },
49  // Another network with more advanced options...

```

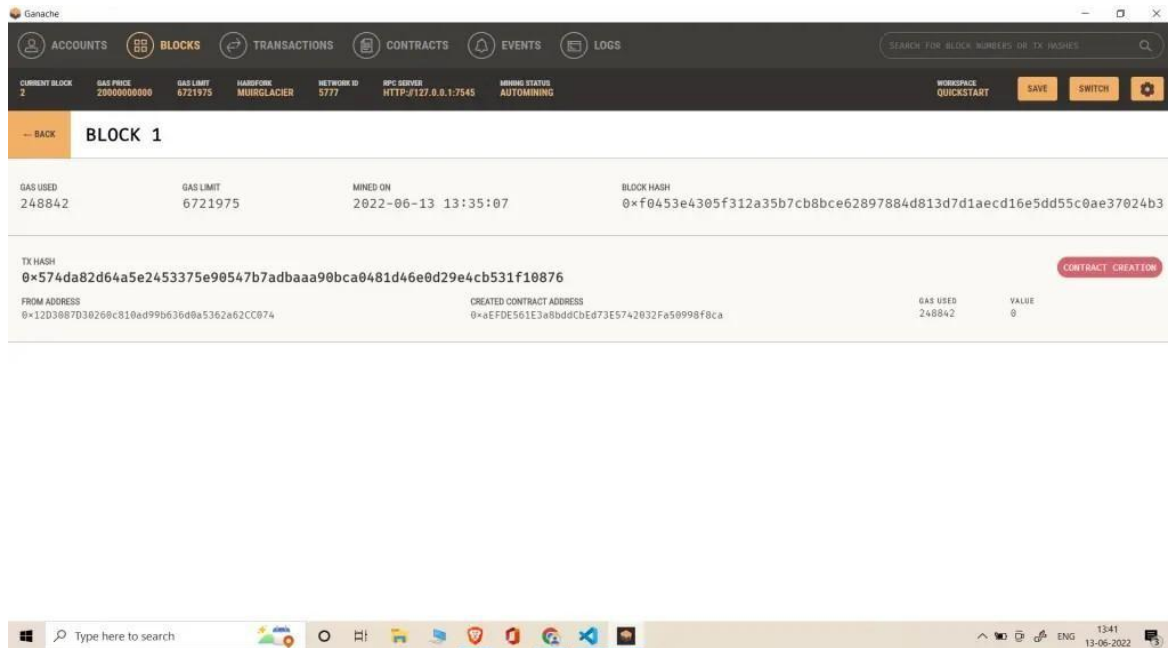
Making changes to truffle-config.js

Run the following command in the terminal:

truffle migrate --reset

Code language: Bash (bash)

This will start the migration process and your smart contract will get deployed to the ganache blockchain. You can check your migrated smart contract on the ganache by going to the block tab of ganache.



Q. How to Test A Smart Contract Using Truffle?

1. Create a test file in the test folder.
2. Use Mocha Framework to test your smart contract.

Run the following command in the terminal to test your smart contract:

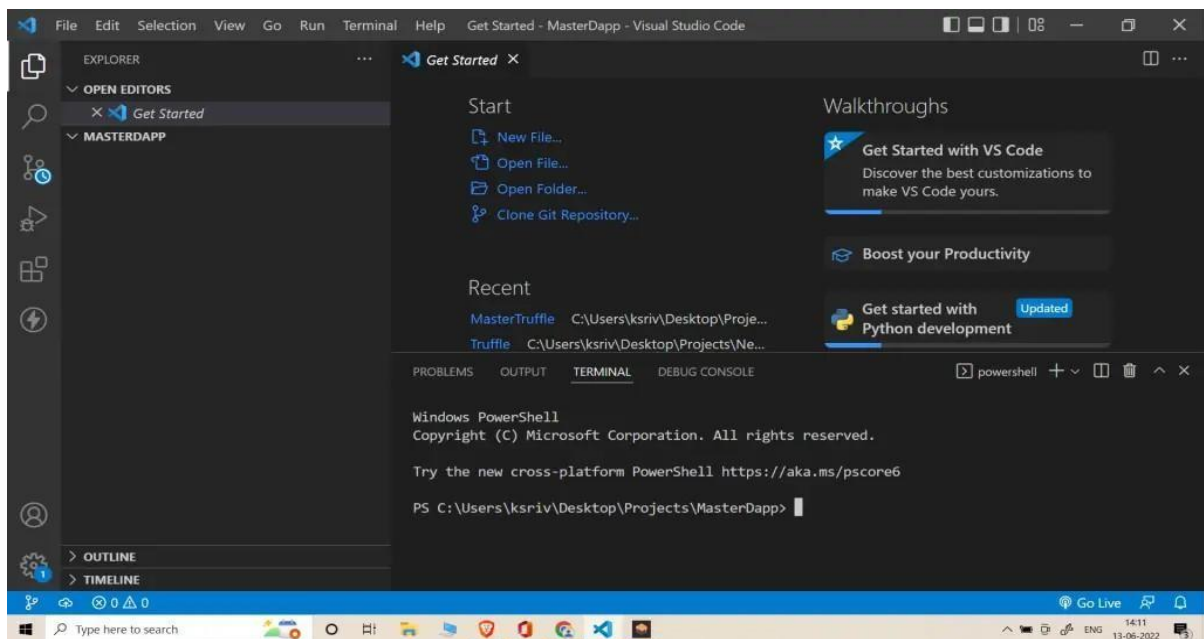
truffle test

Code language: Bash (bash)

Q. How to Create a Decentralized Application (Dapp) Using Truffle?

To create ReactJs Dapp using Truffle let us create a new folder “MasterDapp” in VS code.

Installation of React Truffle Box

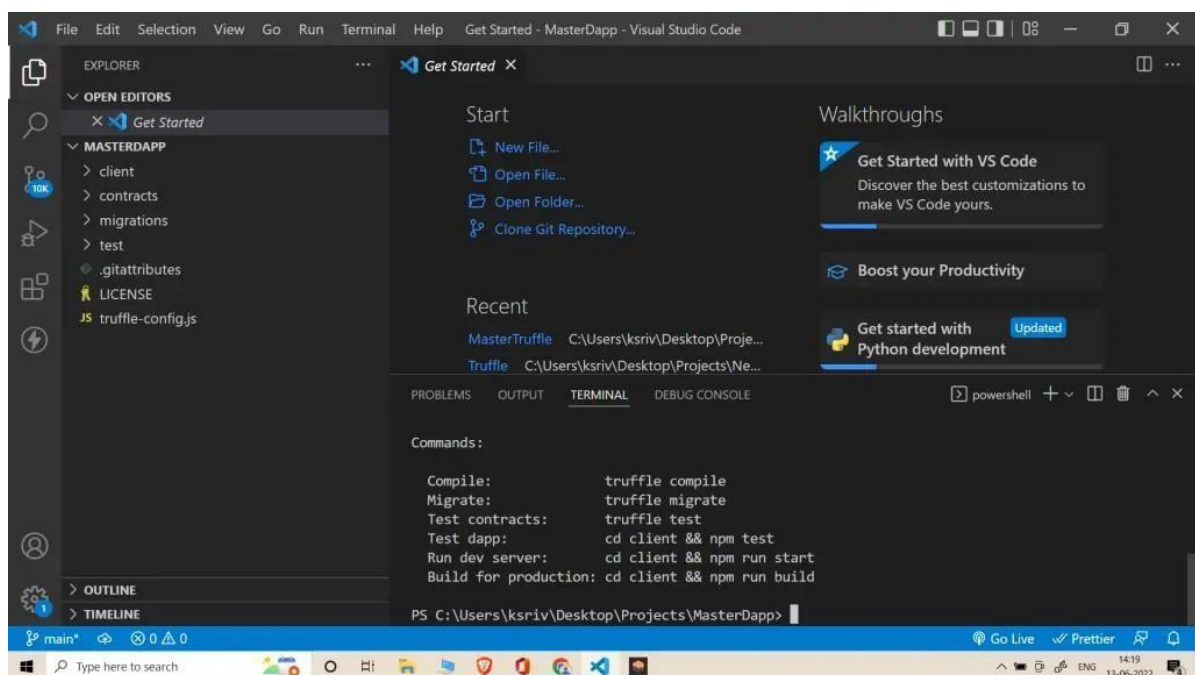


Run the following command in the terminal:

`npx truffle unbox react`

Code language: Bash (bash)

Once you will run the above command you will see a new folder by the name client. client – This folder will consist of the React components
Every other folder such as contract, migration, and test is the same as before.



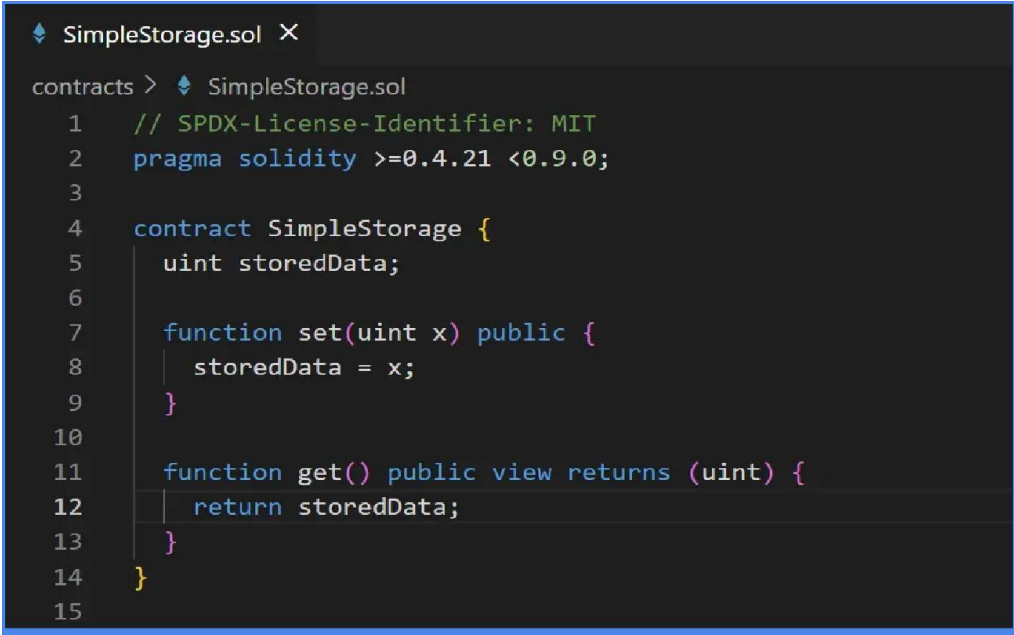
Before running our decentralized application (Dapp) let us make some changes to our truffle- config.js . Copy the following code in the **truffle-config.js** file.

```
const path = require("path"); module.exports = { contracts_build_directory:
path.join( dirname, "client/src/contracts"),
networks: {
development: {
port: 7545,
network_id: "*",
host: "localhost",
}},
compilers: {
solc: {
version: "0.8.9"
}}};
```

1. **contracts_build_directory** – This is to create our build folder inside the client/src/contracts folder. This is done so that our react application can interact with the ABI of our smart contract. By interacting with the ABI of our smart contract we can interact with the smart contract deployed on the blockchain.
2. **networks** – To deploy our smart contract on the Blockchain network we want. In this case, we are deploying on the Ganache Blockchain.
3. **compilers** – This is for the compiler version that we want to use to compile our smart contract.

Sample DApp In React Truffle

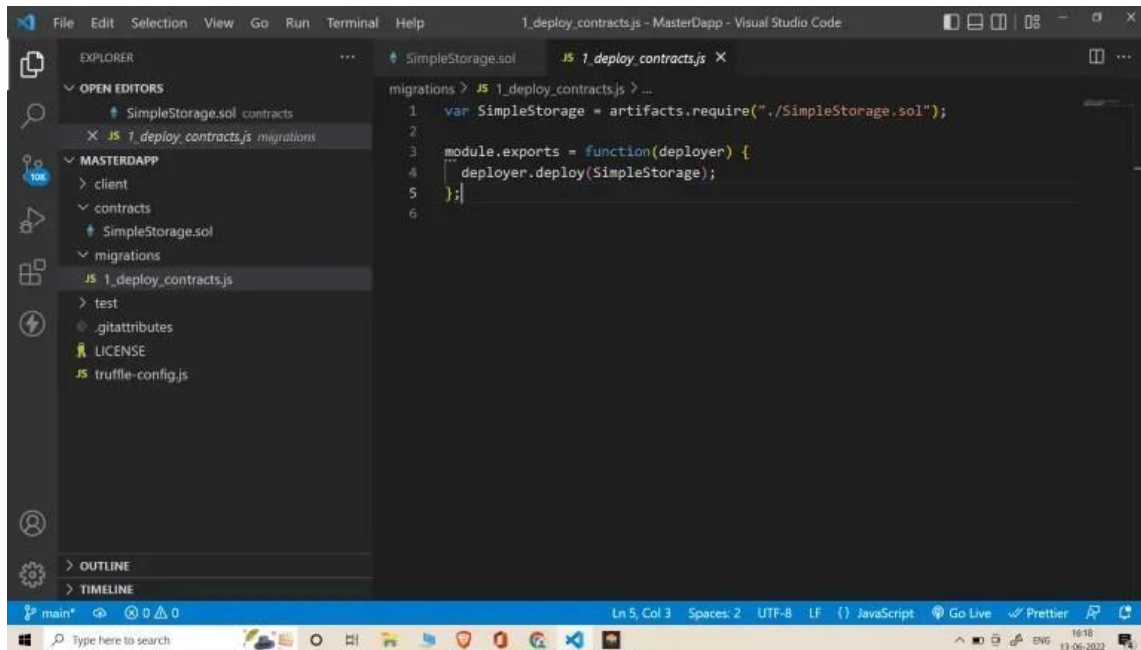
Truffle provide us with a sample Dapp when we installed React Truffle Box. Let us check that. In the contracts folder, you will find a SimpleStorage.sol smart contract. This contract will be used in the sample Dapp provided by the truffle. You can delete the Migration.sol smart contract.



```
SimpleStorage.sol X
contracts > SimpleStorage.sol
1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.4.21 <0.9.0;
3
4  contract SimpleStorage {
5      uint storedData;
6
7      function set(uint x) public {
8          storedData = x;
9      }
10
11     function get() public view returns (uint) {
12         return storedData;
13     }
14 }
15
```

SimpleStorage.sol

Let us go to the migration folder. And delete, 1_initial_migration.js as we will not require this. We will only require the deployment script for the SimpleStorage.sol smart contract. Then change the name of 2_deploy_contract.js to 1_deploy_contract.js.

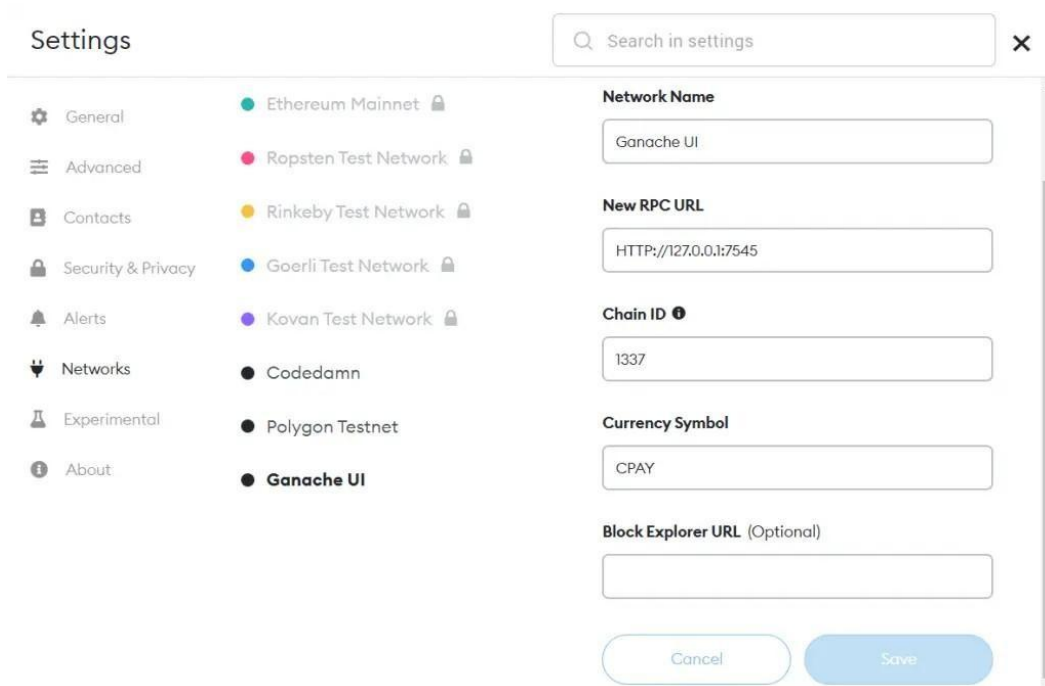


Migration/Deploy script of SimpleStorage smart contract using truffle

- To use the sample Dapp provided by Truffle we need to install Metamask. MetaMask is a software
- cryptocurrency wallet used to interact with the Ethereum blockchain.

You can easily install Metamask from [here](#).

- Once Metamask wallet is installed. Go to Settings -> Network. And click on the “add a network” button.
- And paste the following as given below and save.



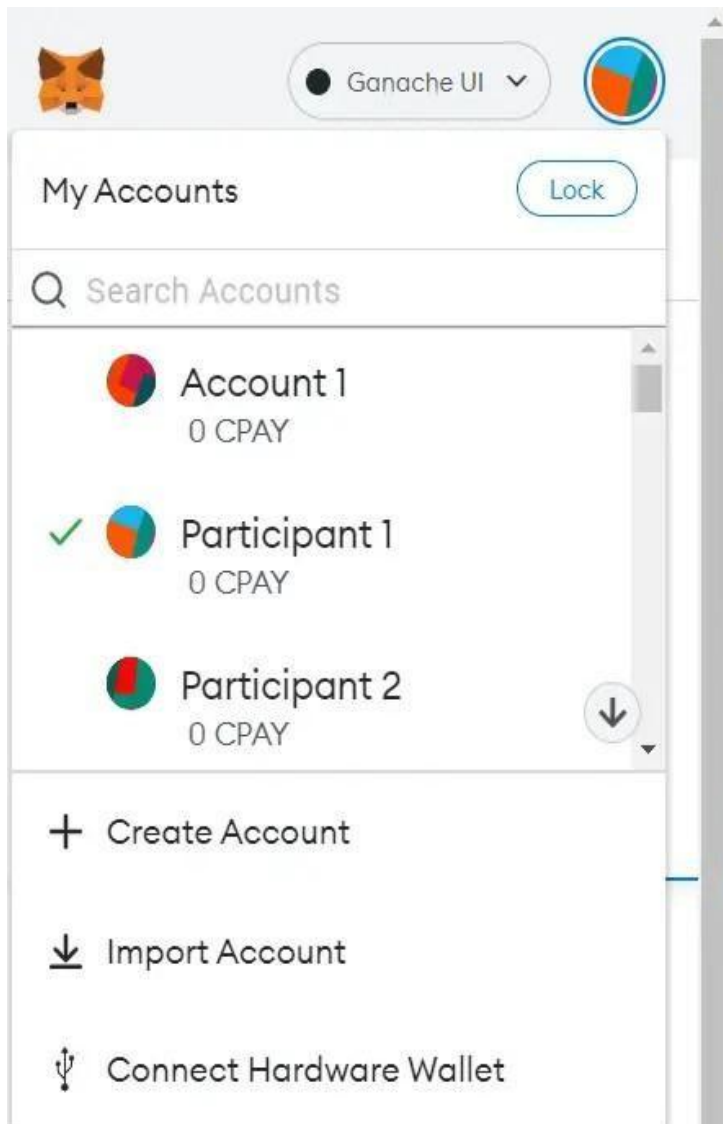
Metamask Network Settings

- We need to import an account from Ganache to the Metamask wallet. In Ganache, click on the “show key” (key like the figure on the right of an account) and copy the private key. You can select any account you want.



Copying the private key

- Let us go to Metamask wallet and import the account by pasting the private key.





Ganache UI



Import Account

Imported accounts will not be associated with your originally created MetaMask account Secret Recovery Phrase. Learn more about imported accounts [here](#)

Select Type

Private Key

Enter your private key string here:

.....

Cancel

Import

Paste the private key copied from Ganache

- If you have done, everything in the correct way, you will see your imported account in the Metamask wallet.



Ganache UI



Not connected

Account 24

0xC72...fD90

99.992 CPAY

Buy

Send

Swap

Assets

Activity

You have no transactions

Need help? Contact [MetaMask Support](#)

Imported Account

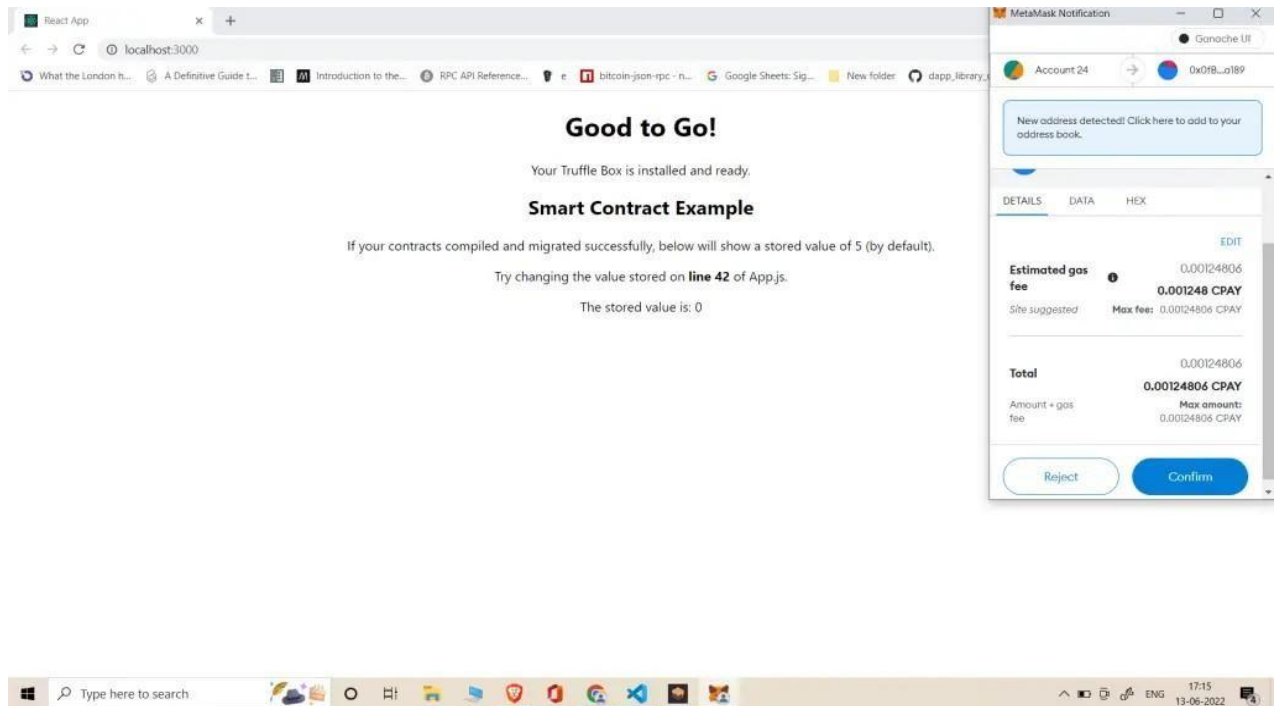
- It's time to deploy our Dapp. First, we will deploy our SimpleStorage.sol smart contract.
- Run the following command to deploy:

truffle migrate --reset

Code language: Bash (bash)

- Split our terminal of VS code. And change the directory to the client and run npm start by the following command. This will start your react application.

cd client npm start



Working Dapp

The **App.js code** is given in the **client/src/App.js** is old React JS code written in the class component. Here, is the new React JS code written in ES6.

```
import React, { useState, useEffect } from "react";
import getWeb3 from "../getWeb3";
import SimpleStorage from "../contracts/SimpleStorage.json";
import "../App.css";
```

```
const App = () => {
  const [state, setState] = useState({
    web3: null,
    contract: null,
  });
  const [storageValue, setStorageValue] = useState(null);
```

```
  useEffect(() => {
    const init = async () => {
      try {
```

```

const web3 = await getWeb3();

const networkId = await web3.eth.net.getId();
const deployedNetwork = SimpleStorage.networks[networkId];
console.log (deployedNetwork.address);
const instance = new web3.eth.Contract(
SimpleStorage.abi,
deployedNetwork && deployedNetwork.address
);

setState({ web3, contract: instance });
} catch (error) {
alert ("Failed to load web3, accounts, or contract Check console for details.");
console.error(error);
}};
init();
}, []);

useEffect(() => {
async function getValue() {
const {contract} = state;
const value = await contract.methods.get().call();
setStorageValue(value);
}
state.contract && getValue();
}, [state,state.contract]);

return (<div className="App">
    <div>The stored value is: {storageValue}</div>
    </div>);
};

export default App;

```

Q. How Do I Start Learning Truffle and Web3?

The codedamn web3 learning path is an excellent place to start learning blockchain and web3 related concepts in depth. We cover truffle, ganache, and even newer tools like hardhat in depth in the courses found there.

Conclusion: We have learned so many things such as how to compile, migrate, and test a smart contract. Not only this but also how to deploy a decentralized application (Dapp) over the ganache blockchain.